

Benchmark Report: pyflate

A small link budget, a compressed payload, and a lot of work between the bits

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Somewhere between a spacecraft and a ground station there is a link budget measured in kilobits.

Compression makes the most of it; decompression turns the arriving bits back into useful data. This benchmark gives that quiet half of the job to Python: read a bit stream, recover symbols, and undo several transformations. Its Huffman matcher looks like the obvious place to start. Measuring it led us to improvements across the whole decoding pipeline, then a Rust extension that executes the symbol decoder through one call per block.

1. Workload and result

Despite its name, the measured pyflate workload is **bzip2**, not DEFLATE. Each iteration decompresses `interpreter.tar.bz2` from **67,562 bytes to 399,360 bytes**; the harness checks the original MD5 outside the timer. The Python tier uses the standard library rather than calling a native decompression library. Its main structures are an integer bit buffer, Huffman tables, a move-to-front (MTF) alphabet, Burrows-Wheeler transform (BWT) index vectors and output buffers.

One block, six Huffman tables, 148,271 symbols. The decoder switches tables every 50 symbols. Huffman decoding, move-to-front and RUNA/RUNB expansion produce a 336,184-byte L-vector. Inverse BWT and final RLE4 expansion produce the 399,360-byte output.

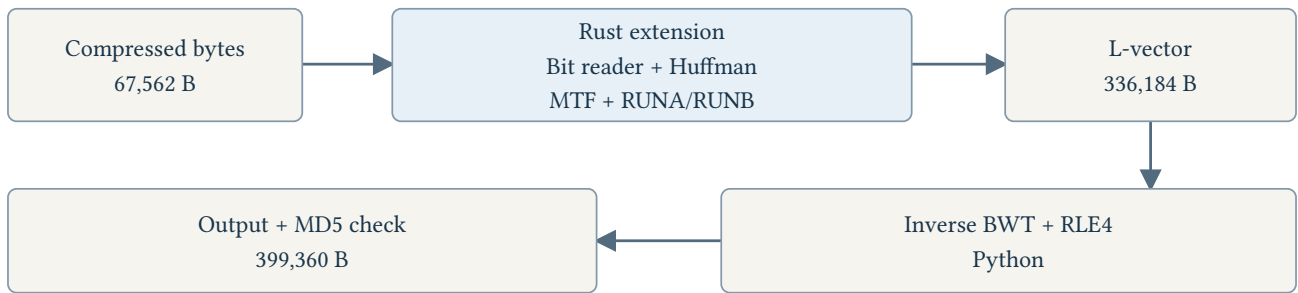


Figure 1: The bzip2 path. RUNA/RUNB and move-to-front are handled together in the symbol loop. The Rust kernel replaces that boundary in one call per block, and it is the same boundary the two accelerators implement.

Configuration	Mean ± SD	vs stock
Stock Python	1,123.49 ± 10.99 ms	1.00×
Optimized Python	281.16 ± 3.05 ms	4.00×
Python + Rust extension	170.01 ± 2.30 ms	6.61×

Our final software path is 6.61× faster than stock: an 84.9% runtime reduction. Python improvements provide 4.00×; over the same benchmark’s Python back end, the Rust symbol decoder adds **1.67×** (Section 4). Both tiers exceed the course’s 7% improvement requirement and match `bz2.decompress` byte for byte, passing the unchanged MD5 check. Rust handles the blue stage above; block headers, inverse BWT and RLE4 remain in Python.

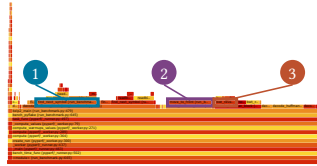
Measurement scope. Course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12; 120 values per configuration, pinned to guest CPU 0, from revision 2c8c754 through the documented scripts and its own Rust build. Source: `results/vm_canonical_20260910_2c8c754/suite/` (Appendix A7). Development profiles and ablations are labeled separately.

Appendix A5 gives the timing distributions and worker-clustering analysis. The repeated measurements support a large improvement on this input; one bzip2 fixture does not establish the same speedup for every file, compression setting or interpreter.

2. Profiling: the whole decoding pipeline

The stock Python-frame profile highlights Huffman decoding and bit-buffer work, with move-to-front and inverse BWT alongside them. Development instrumentation found a mean of about five entries visited by the symbol matcher, despite its table capacity of 258. A lookup table can remove this scan, but the measured scan is short; per-symbol interpreter work and the later transformations also deserve attention.

Full recorded flame graph

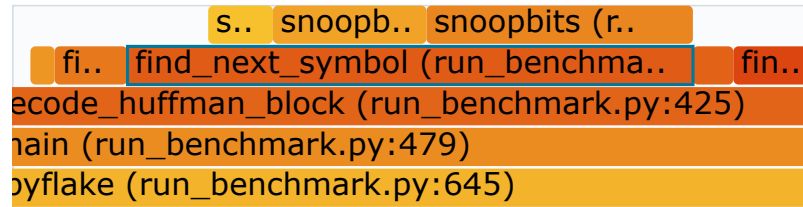


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
py-spy emits one frame per
source line, so a function is
many boxes; shares are totals.

Highlighted regions - enlarged with surrounding frames

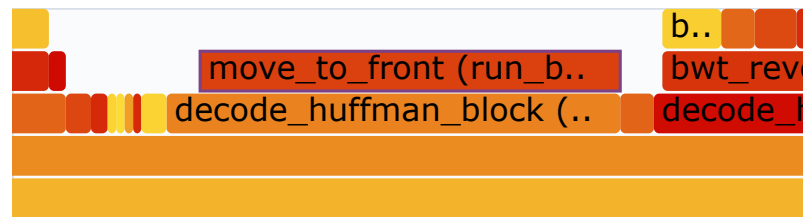
1. Huffman matcher

find_next_symbol | 40.87% inclusive, across 7 per-line frames; widest 18.82% outlined



2. Alphabet updates

move_to_front | 13.44% inclusive



3. Inverse BWT

bwt_reverse | 14.52% inclusive, across 5 per-line frames; widest 6.18% outlined

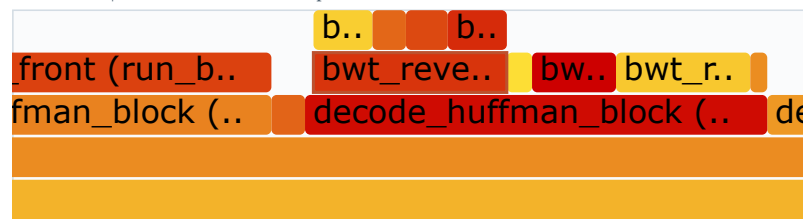


Figure 2: Full stock Python-frame flame graph, retaining startup and harness context. Numbered outlines identify exactly the call paths enlarged on the right. Inclusive percentages use the original whole-profile denominator; nested shares overlap.

Function	Stock self	Stock incl.	Opt. self	Opt. incl.
decode_huffman_block	26.08%	97.31%	1.64%	82.79%
_decode_symbols_python	–	–	32.79%	32.79%
find_next_symbol	12.63%	40.86%	–	–
move_to_front	13.44%	13.44%	0.82%	0.82%
snoopbits	9.41%	16.13%	–	–
readbits	9.95%	13.44%	2.46%	2.46%
_mask	5.91%	5.91%	–	–
bwt_reverse	8.33%	14.52%	13.11%	38.52%
bwt_transform	6.18%	6.18%	25.41%	25.41%
rle4_expand	–	–	5.74%	5.74%

Self excludes callees; **inclusive** includes them. Each profile has its own denominator, so percentages locate remaining work rather than measure absolute speedup. The optimized symbol loop incorporates the old matcher, bit reader and most MTF handling; residual readbits / move_to_front calls parse header selectors. Figure labels aggregate a function's per-line frames, matching the table within rounding; outlines mark one frame.

Symbol decoding is the largest optimized function (32.79% self), while whole inverse BWT accounts for 38.52% inclusive. Both matter for the next offload decision. These sparse profiles identify candidates; Section 4 explains why they cannot supply a precise Amdahl fraction for the separately measured native timing pair.

3. Changes that shipped

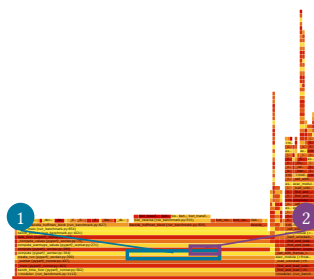
- **Reduce per-byte overhead:** read the compressed input once, refill the bit window in chunks, bind hot values to locals and accumulate bytes without allocating an object per byte.
- **Decode canonical Huffman codes:** use length-indexed tables and a primary lookup table with a fallback for longer codes. Table construction is amortized over the block.
- **Improve the back end:** replace comparison sorting with counting-sort BWT construction; reverse the MTF list so updates move $O(\text{rank})$ entries; use regex-assisted RLE4 expansion to reduce Python loop iterations. The byte scan/output work is still linear overall.

Component reverted from T3	VM trial 1	VM trial 2	Dev, 3.12
None (full T3 runtime)	271.2 ms	273.9 ms	175.5 ms
Regex-assisted RLE4	+100.6 ms	+98.7 ms	+32.3 ms
Primary Huffman lookup	+52.6 ms	+49.4 ms	+10.1 ms
Counting-sort BWT	+20.8 ms	+17.2 ms	+38.6 ms

Ablation scope. Each row puts one stock component back into the optimized Python decoder (dev/pyflate/ablate.py, which drives the T3 module directly rather than the pyperf harness) and reports the added time, best of seven interleaved decompressions. Costs need not add. The two VM trials are separate runs on the course VM, release CPython 3.10.12, pinned to one guest CPU; the last column is the earlier development run on Windows / CPython 3.12.6 (dev/pyflate/FINDINGS.md §2b).

The measured platform reverses part of the development ranking. On the VM, regex-assisted RLE4 is worth about 100 ms and the primary Huffman lookup about 50 ms – both more than counting-sort BWT at about 20 ms – and the two trials agree within 4 ms on every row. The development run on 3.12 had ranked BWT first and the Huffman table last, so the ordering is a property of the interpreter and platform, not of the algorithms. What survives on both is that no single change dominates: the pipeline, not the matcher alone, had to be optimized. Raw output: results/vm_rerun_20260910_3697a63/.

Full recorded flame graph

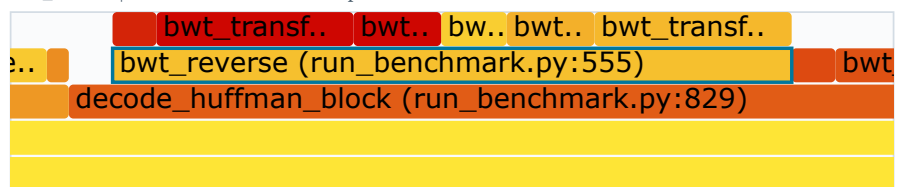


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
py-spy emits one frame per
source line, so a function is
many boxes; shares are totals.

Highlighted regions - enlarged with surrounding frames

1. Inverse BWT

bwt_reverse | 38.53% inclusive, across 4 per-line frames; widest 25.41% outlined



2. Index-table construction

bwt_transform | 25.42% inclusive, across 6 per-line frames; widest 7.38% outlined

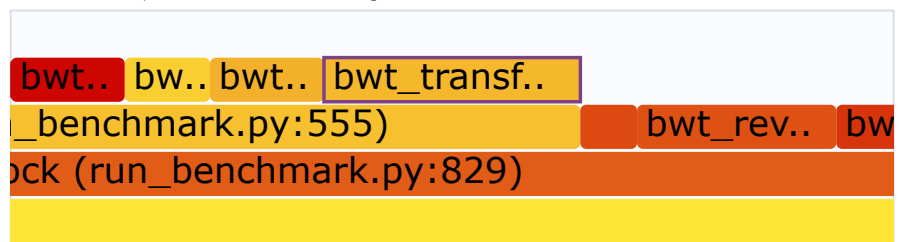


Figure 3: Full optimized **Python** profile, before native offload. Inverse BWT and its index-table construction remain visible beside `_decode_symbols_python`, the single symbol-decode loop that replaced the stock matcher and bit-reader frames.

Worked examples of the shipped transformations

Before/after: removing the per-symbol search. The stock matcher visits Huffman entries and calls the bit reader for candidate lengths. The optimized loop peaks once into a flat table; a nonzero entry packs the symbol and consumed length. These excerpts summarize the control flow, omitting refill, table switches and output handling.

Stock matcher (schematic)

```
for entry in table:
    code = field.snoopbits(entry.bits)
    if code == entry.code:
        field.readbits(entry.bits)
        return entry.symbol
```

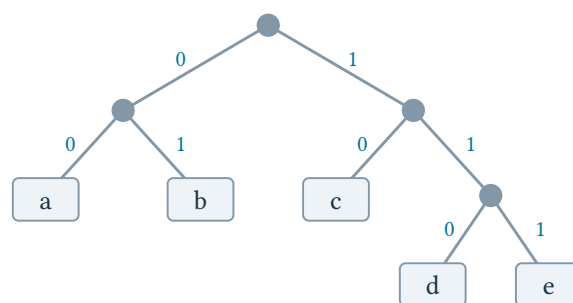
Optimized lookup (abbreviated)

```
v = tbl[peek(pb)]
if v:
    consume(v & 31)
    symbol = v >> 5
else:
    # Extend bits using limit/base/perm.
    symbol = canonical_fallback()
```

peek, consume and canonical_fallback are explanatory names; the shipped loop performs these operations on local integers. Source: build_huffman_table and _decode_symbols_python in benchmarks/bm_pyflate/run_benchmark.py.

Example codes

```
a = 00
b = 01
c = 10
d = 110
e = 111
```



Canonical ranges

len 2: 00..10

len 3: 110..111

Match a length range,
then index the symbol.

Example: 110 decodes to d.

Figure 4: Illustrative canonical Huffman codes. Codes of equal length are consecutive; length-indexed ranges identify the symbol without scanning individual candidates. The optimized implementation adds a primary lookup for short codes.

A four-entry lookup, step by step. Use the illustrated code lengths [2, 2, 2, 3, 3] and temporarily choose $pb = 2$ (the shipped default is 11, capped by the longest code). The primary entries for 00, 01, 10, 11 are (a, 2), (b, 2), (c, 2), and fallback. On input 01110, peek 01, output b and consume two bits; 110 remains. Peek 11: its entry is zero, so extend to three bits. Now code 6 lies at or below $limit[3] = 7$; $base[3] = 3$, hence $perm[6 - 3] = perm[3] = d$. Consume three bits. Thus the stream decodes to b,d without searching individual symbols. With a three-bit primary table, 00x, 01x and 10x each duplicate a two-bit code into two slots; the stored length still consumes only two bits, preserving the next symbol's first bit.

Counting-sort BWT construction. For $L = \text{banana}$, counts are a:3, b:1, n:2. Prefix sums place their buckets at offsets 0, 3 and 4. Scanning L left to right stores each original index in the next slot of its bucket, giving $T = [1, 3, 5, 0, 2, 4]$. Equal symbols keep their original order. The stock path sorts L and searches for bucket starts; the optimized path counts 256 possible byte values and fills T in $O(n + 256)$ work. This constructs the traversal table; inverse BWT still follows $end = T[end]$ once per output byte.

Reversed move-to-front. Start with logical order [a, b, c, d], rank 2 selecting c. The stock slicing update produces [c, a, b, d] by rebuilding the list. The optimized physical list is reversed, [d, c, b, a]. The non-run symbol is $r = rank + 1 = 3$; $pop(-3)$ returns c, then $append(c)$ leaves [d, b, a, c], the reverse of the same logical answer. The pop shifts only two trailing entries and append places c at the logical front. This preserves the operation while moving $O(rank)$ entries instead of rebuilding the whole alphabet.

RLE4 remains a separate final stage. Four equal bytes followed by count k represent $4+k$ copies. For example, [A, A, A, A, 2, B] expands to six A bytes and one B. The implementation's regex locates runs in C and bulk byte operations construct the repeated span; it does not change the bzip2 representation or the linear size of the output.

4. Native execution: what improves, what remains

Our Rust/PyO3 BlockDecoder handles the bit reader, Huffman decode, MTF and RUNA/RUNB expansion. Header parsing, inverse BWT, RLE4 and MD5 remain in Python. One block-level call returns the L-vector and the ending bit position; checking both catches output errors and stream desynchronization.

Same benchmark file	Mean ± SD	vs Python
Python backend	283.88 ± 3.34 ms	1.00×
Native decode backend	170.01 ± 2.30 ms	1.67×

The native row is the same measurement as the Python + Rust row on page 1. The Python row runs the same benchmark file with its Python back end directly under pyperf, not through pyperformance, so it differs slightly from page 1's optimized Python; both cover the complete benchmark. `HWSW_BACKEND=python|native|auto` selects the implementation; native mode requires the extension, while auto mode falls back. The benchmark forwards the variable to pyperf's workers itself, so a scrubbed worker environment cannot quietly change which back end is measured.

The submitted Rust source was rebuilt on the VM. Eleven crate tests and seven blocks across five Python/bz2 fixtures passed, including exact ending bit positions; the eleventh is a property test that decodes random Kraft-complete Huffman tables against an independently written canonical encoder. A dispatch check confirmed that native and auto load the new extension and call it once per block.

Matched-work CPU counters

These are medians of three warm-loop VM runs, each decoding the input 16 times, divided by 16. Both backends produced the same output digest. They are a separate experiment from the rigorous timing table above, with acknowledged counter gating after setup (Appendix A2).

Metric per complete decode	Python	Hybrid native
Elapsed time	288.68 ms	174.44 ms
Instructions	1,947.94 M	1,171.65 M
Cycles	675.92 M	404.24 M
IPC	2.88	2.90
Branches	349.43 M	207.30 M
Branch misses (rate)	544,260 (0.156%)	301,458 (0.145%)
Generic cache references	1.749 M	1.686 M
Generic cache misses (rate)	69,322 (3.964%)	52,842 (3.133%)

Native decoding reduces instructions by **39.9%**, cycles by **40.2%** and branch misses by **44.6%**, while IPC changes little. The approximately 1.65× speedup in this experiment comes mainly from removing interpreter work. Generic cache misses fall **23.8%**, a smaller reduction than instructions: the remaining Python BWT/RLE4 pipeline still processes the whole block. Aggregate counters cannot attribute misses to a particular stage. L1 events are omitted because an earlier capture returned invalid load counts.

Offload limits and the next target

Amdahl's law gives $S = 1 / ((1 - f) + f/s)$ for kernel speedup s and offloaded fraction f of **optimized Python** runtime. Even infinite symbol-decoder speed leaves BWT and RLE4. The sampled 32.79% decoder share is a hotspot locator, not a calibrated value of f for the native timing pair: it would imply a maximum 1.49× gain, below the measured 1.67×. Under the simple unchanged-residual model, 1.67× requires at least 40.1% offloadable work. Separate sampling, setup inside the decoder, and differences between measured runs prevent identifying f from this profile alone. Matched phase timing is needed to explain the ratio quantitatively; the end-to-end measurement itself remains the evidence for the speedup.

Unlike nbod's native integration loop, this offload leaves over a billion instructions per decode in a largely Python pipeline. The next target is the **whole inverse BWT**, including table construction, not just its dependent traversal. Earlier VM phase timings measured **137.7 ms for whole inverse BWT** and **47.8 ms for traversal alone** (Appendix A3). The first includes construction and traversal; the two numbers must not be added. A native port and working-set sweep would test whether memory latency then becomes limiting; the present data do not establish that.

5. Hardware acceleration: implementation and estimates

huffman_engine followed by mtf_cam produces the same L-vector as the Rust kernel, over the same boundary. Software keeps block headers, inverse BWT, RLE4 and MD5. Both modules are implemented in SystemVerilog and have module-level verification and synthesis evidence. Physical-design attempts stopped before routed sign-off. The on-chip chain and platform DMA boundary are the integration design; standalone module results do not establish a measured end-to-end hardware speedup.

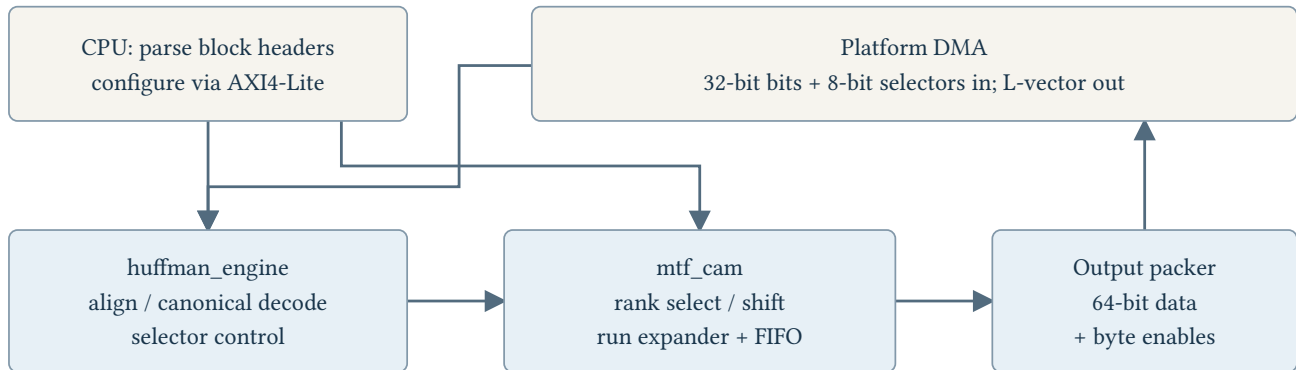


Figure 5: AXI4-Lite configures both modules; AXI4-Stream carries data. The symbol stream stays on chip. Platform DMA returns the L-vector to software.

Huffman engine

A bit aligner feeds a canonical-code comparator/lookup datapath; a selector controller changes the active table every 50 symbols. Inputs are a 32-bit compressed stream and an 8-bit selector stream; outputs are 32-bit beats carrying a 9-bit symbol plus type fields. A 32-bit AXI4-Lite window holds controls, START_BIT, lengths and counters, for six tables and bzip2's 20-bit maximum code length. It decodes the benchmark block in **149,276 cycles** for 148,271 symbols, so **K1 = 1.0068 cycles per symbol** including table construction and switches, and is trace-exact against the golden model over every beat.

MTF and run expansion

The input is a **rank**, so the decoder indexes the alphabet, shifts the preceding entries and expands RUNA/RUNB groups through a buffered output packer at eight bytes wide. It sustains **K3 = 1.0686 cycles per symbol**, and the CAM is 68% of its area.

Measured cost and what it buys

sky130, Yosys + OpenLane	huffman_engine	mtf_cam
Cells	151,058	18,814
Area	1.634 mm ²	0.187 mm ²
Timing-derived frequency	≈ 8.9 MHz	≈ 37.6 MHz
Power estimate	not reported	≈ 13.7 mW
Cycles per symbol	1.0068	1.0686
Directed + random tests	17 / 17	16 / 16
Line / toggle coverage	90.4% / 90.3%	92.0% / 93.8%

Evidence levels. Huffman's 8.9 MHz comes from pre-placement static timing; MTF's 37.6 MHz comes from post-CTS timing. These are different stages of physical estimation, not demonstrated silicon operating frequencies or directly comparable timing closure results. Placement, buffering and routing may change either estimate. Neither completed 50 MHz sign-off. MTF's 13.7 mW is a tool estimate using default switching activity at the run's clock constraint, not measured workload power at 37.6 MHz; it cannot support energy savings. Coverage percentages use the documented exclusions, and MTF invariant proofs are bounded to depth 24 rather than exhaustive. The regression counts are recorded results, not new runs performed for this report revision.

Conditional benefit and integration cost

Use $T_{\text{new}} = (1 - f) \cdot T_{\text{stock}} + T_{\text{hardware}} + T_{\text{interface}}$. The older integration model assumes $f = 0.496$ for Huffman; using the canonical stock mean 1,123.49 ms gives about $1.97\times$ at 50 MHz (149,276 cycles = 2.99 ms), or $1.93\times$ at 8.9 MHz (16.77 ms), before interface cost. The surviving software contributes about 566 ms. These are conditional projections, not measurements of the implemented hardware attached to Python.

MTF's recorded RTL block takes 158,441 cycles (1.0686 cycles/symbol), about 4.21 ms at 37.6 MHz. Its standalone model uses the sampled 13.44% stock MTF share and yields about $1.15\times$ end to end. The older isolated 80.4 ms MTF microbenchmark would imply a stage-only $19.1\times$ ratio at this clock, or $25.4\times$ at the 50 MHz target; it is a different experiment from the whole-decoder profile and must not be substituted for that profile's runtime fraction.

Adding the old Huffman assumption (49.6%) to the newer MTF profile share (13.44%) gives $f \approx 0.63$ and an illustrative ideal ceiling near $2.7\times$. These fractions do not come from a single matched decomposition of the combined boundary, which also includes run expansion. That sum is therefore a sensitivity example, not a measured ceiling for the chained design. A chain comparison should time the exact replaced region and residual in the **same** optimized Python workload, then compare with the already delivered 170.01 ms hybrid path. A stock-based projection cannot demonstrate an improvement over that stronger baseline.

The diagram's chain uses on-chip symbol transfer. A shared clock would be constrained by both modules; separate clocks would require an explicit clock-domain crossing. In either case, use a combined backpressure simulation rather than adding standalone cycles/symbol. Input DMA supplies compressed bits and selectors, and output DMA drains 336,184 L-vector bytes. Configuration, DMA setup, copies and cache maintenance contribute to $T_{\text{interface}}$; transfer time may overlap compute only if buffers and bandwidth sustain it. The recorded module and driver tests do not measure these platform costs.

Area and throughput trade-offs

Huffman storage includes a 17.3 kbit symbol table and an 8.6 kbit length window implemented as flops in this standard-cell flow. SRAM is a proposed alternative: the documented estimate is about 0.75 mm^2 of **remaining standard cells plus two macros**. Macro area, access latency, ports and integration must be established before claiming total area below 1 mm^2 or unchanged throughput. No SRAM version was synthesized here.

MTF output bytes/cycle	Mapped area	Modeled cycles/symbol
4-byte output	0.182 mm^2	1.175
8-byte output (selected)	0.187 mm^2	1.063
16-byte output	0.208 mm^2	1.023

$W = 8$ is the smallest modeled width meeting the 1.10 cycles/symbol goal; $W = 16$ costs 10.9% more area for about 3.9% modeled throughput gain. The W -sweep uses the cycle model; the default RTL's measured 1.0686 includes additional implementation overhead. The CAM's 68% area share explains why reducing packer width saves little total area. Timing must be checked for each implementation rather than assuming identical frequency across widths.

6. Conclusion

The delivered Python-plus-Rust path decompresses the input in **170.01 ms**, achieving **6.61 $\times$ over stock** with byte-exact output. Python improvements provide $4.00\times$ and the native decoder adds $1.67\times$, leaving BWT and RLE4 as the next software targets.

Both accelerators have RTL verification and synthesis evidence. Their end-to-end benefit remains conditional on the software fraction, platform interface and completed physical timing. Inverse BWT and RLE4 remain software work across both offload approaches.

Evidence: `hw/huffman_engine/docs/{ppa,integration}.md`, `hw/mtf_cam/docs/{ppa,integration}.md`.
Methods and profiles: [report_appendix.pdf](#). Repository: [HWSW_Project](#).